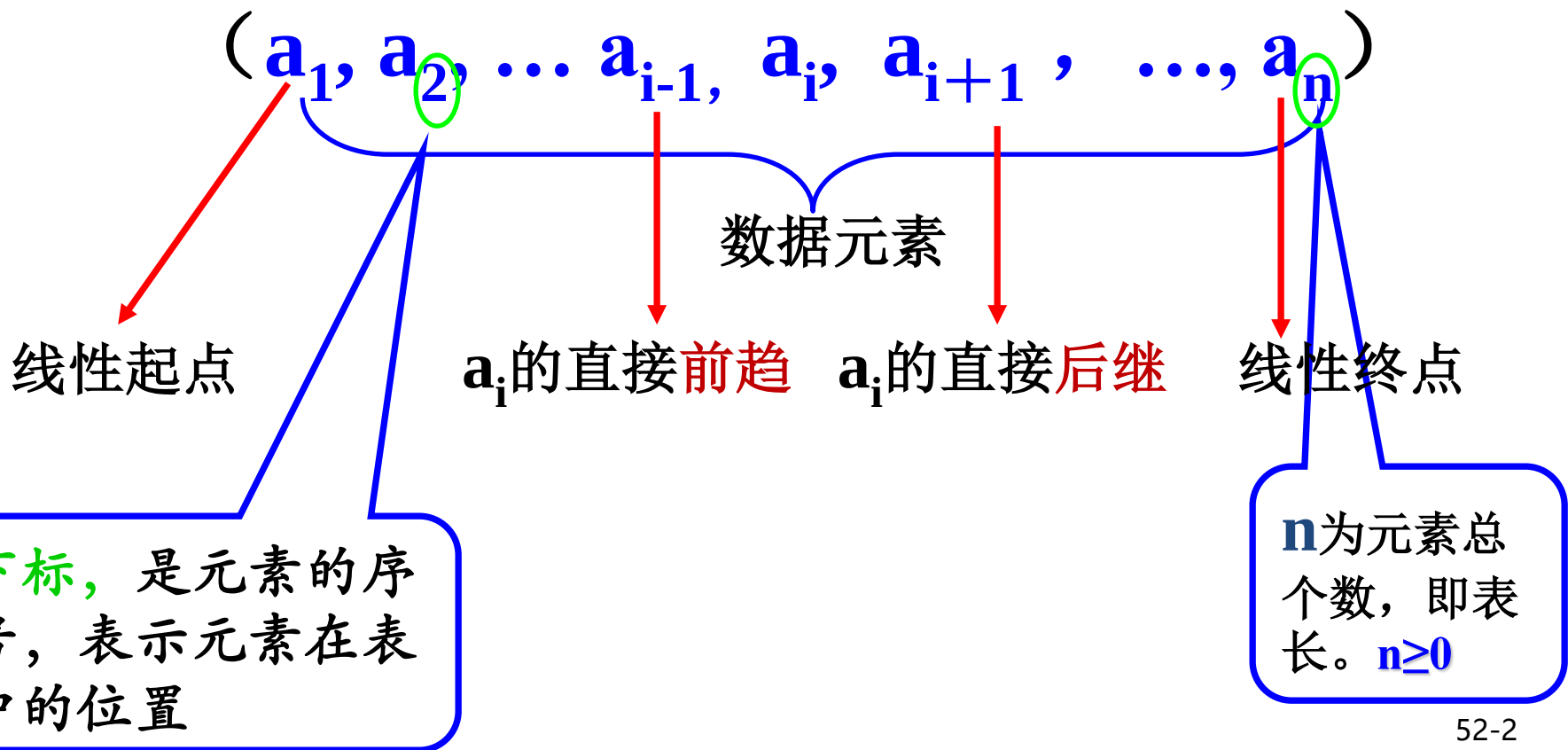


第二章 线性表

回顾

线性表逻辑结构

线性表的定义： 用数据元素的有限序列表示



回顾

线性表的抽象数据类型

ADT List{

数据对象: $D = \{a_i \mid a_i \in \text{ElemSet}, i=1,2,\dots,n, n \geq 0\}$

数据关系: $R1 = \{ \langle a_{i-1}, a_i \rangle \mid a_{i-1}, a_i \in D, i = 2,3,\dots,n \}$

数据操作:

InitList(&L);

ListLength(L);

GetElem(L,i,&e);

LocateElem(L,e);

ListInsert(&L,i,e);

ListDelete(&L,i,e);

.....

}

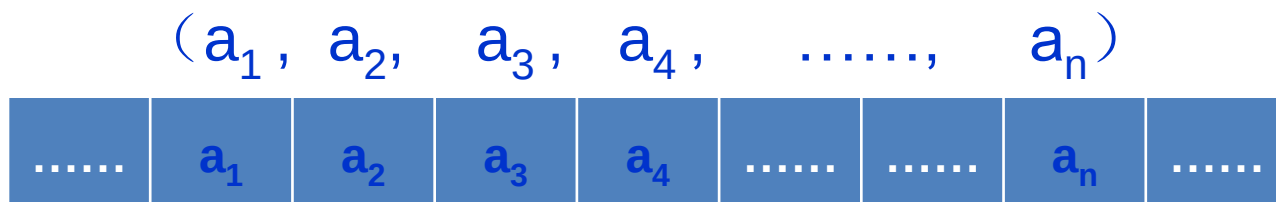
第2章 线性表

- 2.1 线性表的类型定义
- 2.2 线性表的顺序表示和实现
- 2.3 线性表的链式表示和实现
- 2.4 一元多项式的表示及相加

2.2 线性表的顺序表示和实现

线性表的**顺序存储结构**就是：把线性表中的所有元素按照其逻辑顺序依次存储到从计算机存储器中指定存储位置开始的一块连续的存储空间中。

这样,线性表中第一个元素的存储位置就是指定的存储位置, 第 $i+1$ 个元素($1 \leq i \leq n-1$)的存储位置紧接在第 i 个元素的存储位置的后面。



线性表 $\leftrightarrow$ 逻辑结构

顺序表 $\leftrightarrow$ 存储结构

特点：逻辑上相邻的元素，物理上也相邻。

2.2 线性表的顺序表示和实现

假定线性表的元素类型为ElemType,则每个元素所占用存储空间大小(即字节数)为

sizeof(ElemType)

整个线性表所占用存储空间的大小为:

$n * \text{sizeof}(\text{ElemType})$

其中,n表示线性表的长度。

2.2 线性表的顺序表示和实现

下标位置	线性表存储空间	存储地址
0	a_1	$\text{LOC}(a_1)$
1	a_2	$\text{LOC}(a_1) + \text{sizeof}(\text{ElemType})$
$\vdots$	$\vdots$	
$i-1$	a_i	$\text{LOC}(a_1) + (i-1) * \text{sizeof}(\text{ElemType})$
$\vdots$	$\vdots$	
$n-1$	a_n	$\text{LOC}(a_1) + (n-1) * \text{sizeof}(\text{ElemType})$
$\vdots$	$\vdots$	
$\text{MaxSize}-1$	$\vdots$	$\text{LOC}(a_1) + (\text{MaxSize}-1) * \text{sizeof}(\text{ElemType})$

2.2 线性表的顺序表示和实现

$L = \text{sizeof}(\text{ElemType})$

$\text{Loc}(a_{i+1}) = \text{Loc}(a_i) + L$

一般来说，线性表的第*i*个元素 a_i 的存储位置为：

$\text{Loc}(a_i) = \text{Loc}(a_1) + (i-1)*L$

其中 $\text{Loc}(a_1)$ 是线性表的第一个数据元素 a_1 的存储位置，通常称作线性表的**起始位置或基地址**。

2.2 线性表的顺序表示和实现

- 用“物理位置”相邻来表示线性表中数据元素之间的逻辑关系。
- 根据线性表的顺序存储结构的特点，只要确定了存储线性表的**起始位置**，线性表中任一数据元素都可随机存取，所以，线性表的顺序存储结构是一种**随机存取**的存储结构。

2.2 线性表的顺序表示和实现

例：设有一维数组**M**，下标的范围是**0 到 9**，每个数组元素用相邻的**4个字节**存储。存储器按字节编址，设存储数组元素**M[0]**的第一个字节的地址是**98**，则**M[3]**的第一个字节的地址是多少？

110

解：已知地址计算通式为：

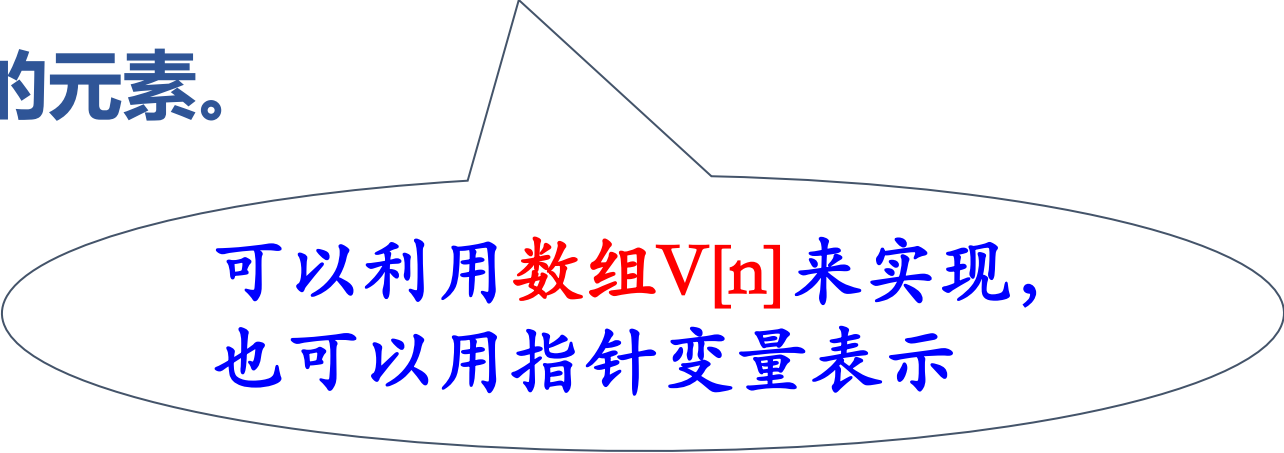
$$LOC(a_i) = LOC(a_1) + L * (i-1)$$

但此题要注意下标起点略有不同：

$$LOC(M[3]) = 98 + 4 \times (4-1) = 110$$

2.2 线性表的顺序表示和实现

顺序存储方法:用一组地址连续的存储单元依次存储线性表的元素。



可以利用数组 $V[n]$ 来实现，
也可以用指针变量表示

数组 → 顺序存储结构。

2.2 线性表的顺序表示和实现

顺序存储：用一组地址连续的存储单元依次存储线性表的元素。

```
#define MaxSize 100

typedef struct {
    ElemType data[MaxSize];
    int length;
} SqList;
```

```
#define LIST_INIT_LENTH 100
#define LISTINCREMENT 10

typedef struct {
    ElemType *elem;
    int length;
    int listsize;
} SqList;
```

```
void *malloc(unsigned int size)
```

```
void *realloc(void *p, unsigned int size)
```

2.2 线性表的顺序表示和实现

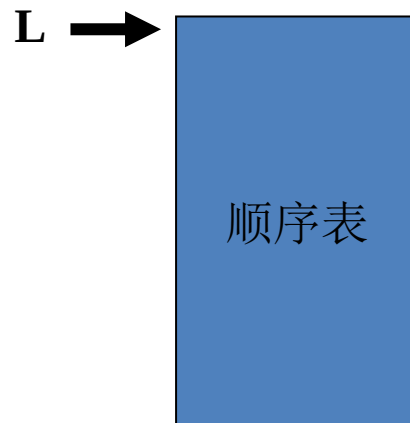
1. 顺序表基本运算算法

(1) 初始化线性表InitList(L)

该运算的结果是构造一个空的线性表L。实际上只需将length成员设置为0即可。

```
void InitList(SqList *&L) //引用型指针{  
    L=(SqList *)malloc(sizeof(SqList));  
    /*分配存放线性表的空间*/  
    if (!L) exit(OVERFLOW);  
    L->length=0;  
}
```

本算法的时间复杂度为 $O(1)$ 。



2.2 线性表的顺序表示和实现

1. 顺序表基本运算算法

(2) 销毁线性表DestroyList(L)

该运算的结果是释放线性表L占用的内存空间。

```
void DestroyList(SqList *&L)
{
    free(L);
}
```

本算法的时间复杂度为 $O(1)$ 。

2.2 线性表的顺序表示和实现

1. 顺序表基本运算算法

(3) 判定是否为空表ListEmpty(L)

该运算返回一个值表示L是否为空表。若L为空表，则返回1，否则返回0。

```
int ListEmpty(SqList *L)
{
    return(L->length==0);
}
```

本算法的时间复杂度为 $O(1)$ 。

2.2 线性表的顺序表示和实现

1. 顺序表基本运算算法

(4) 求线性表的长度ListLength(L)

该运算返回顺序表L的长度。实际上只需返回length成员的值即可。

```
int ListLength(SqList *L)
{
    return(L->length);
}
```

本算法的时间复杂度为 $O(1)$ 。

2.2 线性表的顺序表示和实现

1. 顺序表基本运算算法

(5) 输出线性表DispList(L)

该运算当线性表L不为空时,顺序显示L中各元素的值。

```
void DispList(SqList *L)
{
    int i;
    if (ListEmpty(L)) return;
    for (i=0;i<L->length;i++)
        printf(" %c",L->data[i]);
    printf("\n");
}
```

本算法中基本运算为for循环中的printf语句, 故时间复杂度为:

$O(L \rightarrow \text{length})$ 或 $O(n)$

2.2 线性表的顺序表示和实现

1. 顺序表基本运算算法

(6) 求某个数据元素值 $\text{GetElem}(L, i, e)$

该运算返回L中第 i ($1 \leq i \leq \text{ListLength}(L)$)个元素的值,存放在e中。

```
int GetElem(SqList *L, int i, ElemType &e)
{
    if (i < 1 || i > L->length) return 0;
    e = L->data[i-1];
    return 1;
}
```

本算法的时间复杂度为 $O(1)$ 。

2.2 线性表的顺序表示和实现

1. 顺序表基本运算算法

(7) 按元素值查找LocateElem(L,e) p25~p26算法2.6

该运算顺序查找第1个值域与e相等的元素的位序。若这样的元素不存在,则返回值为0。

```
int LocateElem(SqList *L, ElemType e)
{
    int i=0;
    while (i<L->length && L->data[i]!=e) i++;
    if (i>=L->length)return 0;
    else return i+1;
}
```

本算法中基本运算为while循环中的i++语句,故时间复杂度为:

$O(L \rightarrow \text{length})$ 或 $O(n)$

2.2 线性表的顺序表示和实现

1. 顺序表基本运算算法

(8) 插入数据元素 $\text{ListInsert}(L, i, e)$

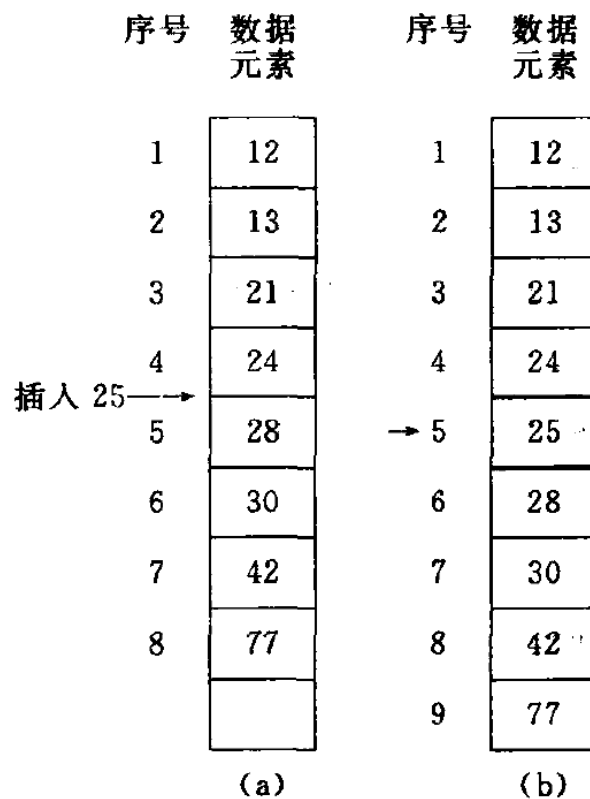
该运算在顺序表 L 的第 i 个位置 ($1 \leq i \leq \text{ListLength}(L) + 1$) 上插入新的元素 e 。

插入前: $L = (a_1, \dots, a_{i-1}, a_i, \dots, a_n)$

插入后: $L = (a_1, \dots, a_{i-1}, \mathbf{b}, a_i, \dots, a_n)$

数据元素 a_{i-1} 和 a_i 之间的逻辑关系发生了变化

2.2 线性表的顺序表示和实现



为了在线性表的第4 和第5 个元素之间插入一个值为25 的数据元素，则需将第5 个至第8 个数据元素依次往后移动一个位置。

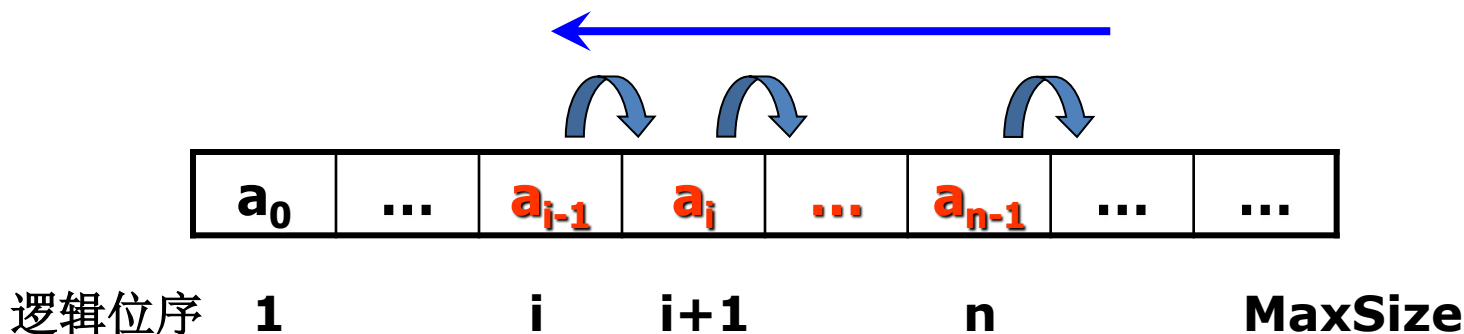
2.2 线性表的顺序表示和实现

算法思想：

- ① 进行合法性检查， $1 \leq i \leq n+1$ ；
- ② 判断线性表是否已满；
- ③ 将第 n 个至第 i 个元素逐一后移一个单元；
- ④ 在第 i 个位置处插入新元素；
- ⑤ 将表的长度加1。

2.2 线性表的顺序表示和实现

```
int ListInsert(SqList *&L,int i,ElemType e) {  
    int j;  
    if (i<1 || i>L->length+1)        return 0;  
    i--; /*将顺序表逻辑位序转化为elem下标即物理位序*/  
    for (j=L->length;j>i;j--) L->data[j]=L->data[j-1];  
        /*将data[i]及后面元素后移一个位置*/  
    L->data[i]=e;  
    L->length++; /*顺序表长度增1*/  
    return 1;  
}
```



2.2 线性表的顺序表示和实现

```
Status ListInsert_sq(SqList &L, int i, ElemType e) {
    if (i<1 || i>L.length+1) return ERROR;
    if (L.length >= L.listsize) {
        newbase=(ElemType*)realloc(L.elem,
                                   (L.listsize+LISTINCREMENT)*sizeof(ElemType));
        if (!newbase) exit(OVERFLOW);
        L.elem = newbase;
        L.listsize+=LISTINCREMENT;
    }
    q=&(L.elem[i-1]);
    for (p=&(L.elem[L.length-1]); p>=q; --p)
        *(p+1) = *p;
    *q=e;
    ++L.length;
    return OK;
}
```

2.2 线性表的顺序表示和实现

算法的复杂度。

- **所需移动结点的次数不仅依赖于表的长度，而且还与插入位置有关。**
- **当 $i=n+1$ ，最好情况，其时间复杂度 $O(1)$ ；**
- **当 $i=1$ 时，结点后移语句将循环执行 n 次，需移动表中所有结点，这是最坏情况。**

2.2 线性表的顺序表示和实现

□ 时间复杂度为 $O(n)$ 。

由于插入可能在表中任何位置上进行，因此需分析算法的平均复杂度

在长度为 n 的线性表中第 i 个位置上插入一个结点，令 $E_{in}(n)$ 表示移动结点的期望值（即移动的平均次数），则
在第 i 个位置上插入一个结点的移动次数为 $n-i+1$ 。故

$$E_{in} = \sum_{i=1}^{n+1} p_i (n - i + 1)$$

2.2 线性表的顺序表示和实现

假设在表中任何位置($1 \leq i \leq n+1$)上插入结点的机会均等, 则

$$p_1 = p_2 = p_3 = \dots = p_{n+1} = 1/(n+1)$$

因此, 在等概率插入的情况下,

$$E_{in} = \sum_{i=1}^{n+1} p_i (n - i + 1) = \frac{1}{n+1} \sum_{i=1}^{n+1} (n - i + 1) = \frac{n}{2}$$

虽然 $E_{is}(n)$ 中 n 的系数较小, 但就数量级而言, 它仍然是线性阶的。因此算法的平均时间复杂度为 $O(n)$ 。

$$T(n) = O(n)$$

2.2 线性表的顺序表示和实现

本算法中注意以下问题：

(1) 顺序表中数据区域有MAXSIZE个存储单元，所以在向顺序表中做插入时先检查表空间是否满了，在表满的情况下不能再做插入，否则产生溢出错误。

(2) 要检验插入位置的有效性，这里 i 的有效范围是： $1 \leq i \leq n+1$ ，其中 n 为原表长。

(3) 注意数据的移动方向。

2.2 线性表的顺序表示和实现

(9) 删除数据元素 ListDelete(L,i,e)

删除顺序表L中的第 i ($1 \leq i \leq \text{ListLength}(L)$)个元素。

删除前: $L = (a_1, \dots, a_{i-1}, a_i, a_{i+1}, \dots, a_n)$

删除后: $L = (a_1, \dots, a_{i-1}, a_{i+1}, \dots, a_n)$

序号	数据元素	序号	数据元素
1	12	1	12
2	13	2	13
3	21	3	21
删除 24 → 4	24	4	28
5	28	5	30
6	30	6	42
7	42	7	77
8	77		

(a) (b)

为了删除第4个数据元素，必须将从第5个至第8个元素都依次往前移动一个位置。

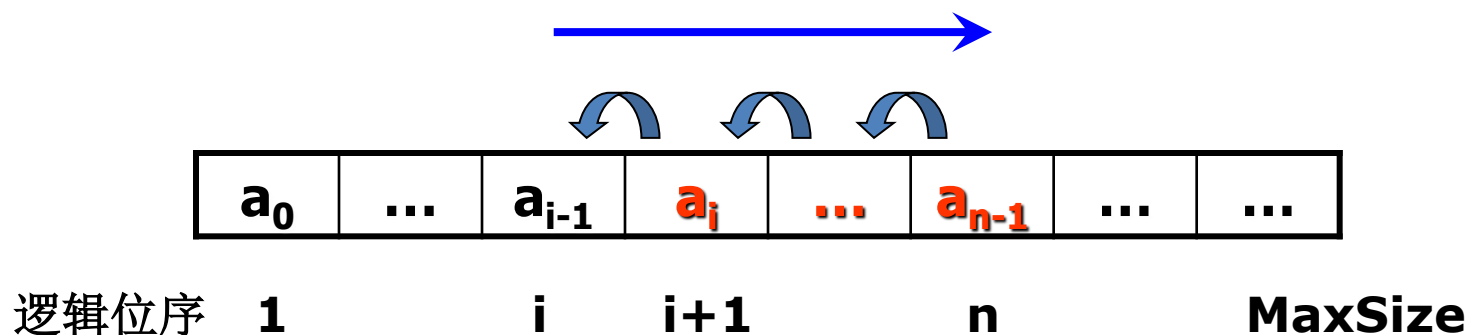
2.2 线性表的顺序表示和实现

算法思想：

- ① 进行合法性检查， i 是否满足 $1 \leq i \leq n$;
- ② 判线性表是否已空， $v.last=0$;
- ③ 将第 $i+1$ 至第 n 个元素逐一向前移一个位置;
- ④ 将表的长度减1。

2.2 线性表的顺序表示和实现

```
int ListDelete(SqList *&L,int i,ElemType &e) {  
    int j;  
    if (i<1 || i>L->length) return 0;  
    i--;    /*将顺序表逻辑位序转化为elem下标即物理位序*/  
    e=L->data[i];  
    for (j=i;j<L->length-1;j++) L->data[j]=L->data[j+1];  
        /*将data[i]之后的元素后前移一个位置*/  
    L->length--; /*顺序表长度减1*/  
    return 1;  
}
```



2.2 线性表的顺序表示和实现

```
Status ListDelete_sq(SqList &L,int i,ElemType &e) {  
    if (i<1||i>L.length) return ERROR;  
    p=&(L.elem[i-1]);  
    e=*p;  
    q=L.elem+L.length-1; //表尾元素结点  
    for (++p;p<=q;++p) *(p-1)=*p;  
    --L.length;  
    return OK  
}
```

- 需将第 $i+1$ 至第 $L.length$ 个元素向前移动一个位置

2.2 线性表的顺序表示和实现

时间复杂度分析:最坏情况是删除第1个元素, 此时要前移 $n-1$ 个元素, 因此: $T(n) = O(n)$

本算法注意以下问题:

(1) 删除第 i 个元素, i 的取值为 $1 \leq i \leq n$, 否则第 i 个元素不存在, 因此, 要检查删除位置的有效性。

(2) 当表空时不能做删除, 因表空时 $L \rightarrow \text{last}$ 的值为 -1 , 条件 $(i < 1 \parallel i > L \rightarrow \text{length} + 1)$ 也包括了对表空的检查。

(3) 删除 a_i 之后, 该数据已不存在, 如果需要, 先取出 a_i , 再做删除。

2.2 线性表的顺序表示和实现

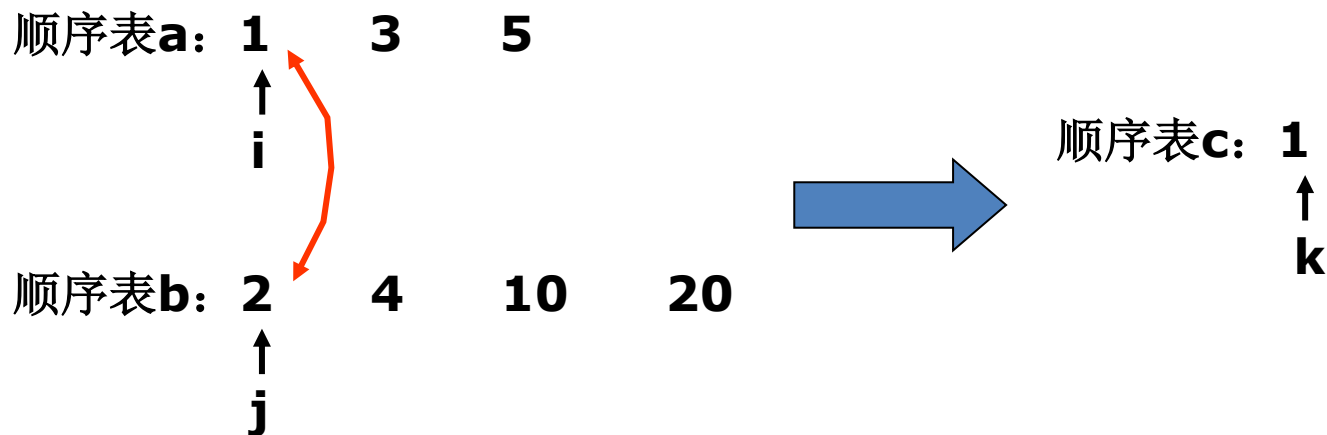
(10)线性表合并 (L_a 、 L_b 为按顺序排列线性表)

将两个元素有序(从小到大)的顺序表合并成一个有序顺序表。

思路:

将两个顺序表进行二路归并。依次扫描通过A和B的元素，比较当前的元素的值，将较小值的元素赋给C，如此直到一个线性表扫描完毕，然后将未完的那个顺序表中余下部分赋给C即可。C的容量要能够容纳A、B两个线性表相加的长度。

2.2 线性表的顺序表示和实现



归并到顺序表c中 ← k记录c中元素个数

1(i=0) ⇔ 2(j=0) ⇒ 将1(i=1)插入r(k=1)

3(i=1) ⇔ 2(j=0) ⇒ 将2(j=1)插入r(k=2)

3(i=1) ⇔ 4(j=1) ⇒ 将3(i=2)插入r(k=3)

5(i=2) ⇔ 4(j=1) ⇒ 将4(j=2)插入r(k=4)

5(i=2) ⇔ 10(j=2) ⇒ 将5(j=3)插入r(k=5)

将b中余下元素插入c中。

2.2 线性表的顺序表示和实现

(10)线性表合并 (La、Lb为按顺序排列线性表)

```
void MergeList(SqList La, SqList Lb, SqList &Lc) {  
    pa=La.elem; pb=Lb.elem;  
    Lc.listsize = Lc.length = La.length+Lb.length;  
    pc=Lc.elem=(ElemType*)malloc(Lc.listsize*sizeof(ElemType));  
    if (!Lc.elem) exit(OVERFLOW);  
    pa_last=La.elem+La.length-1;  
    pb_last=Lb.elem+Lb.length-1;  
    while (pa<=pa_last&&pb<=pb_last) {  
        if(*pa<=*pb) *pc++=*pa++;  
        else *pc++=*pb++; }  
    while (pa<=pa_last) *pc++=*pa++;  
    while (pb<=pb_last) *pc++=*pb++;  
}
```

时间复杂度
 $O(La.length + Lb.length)$

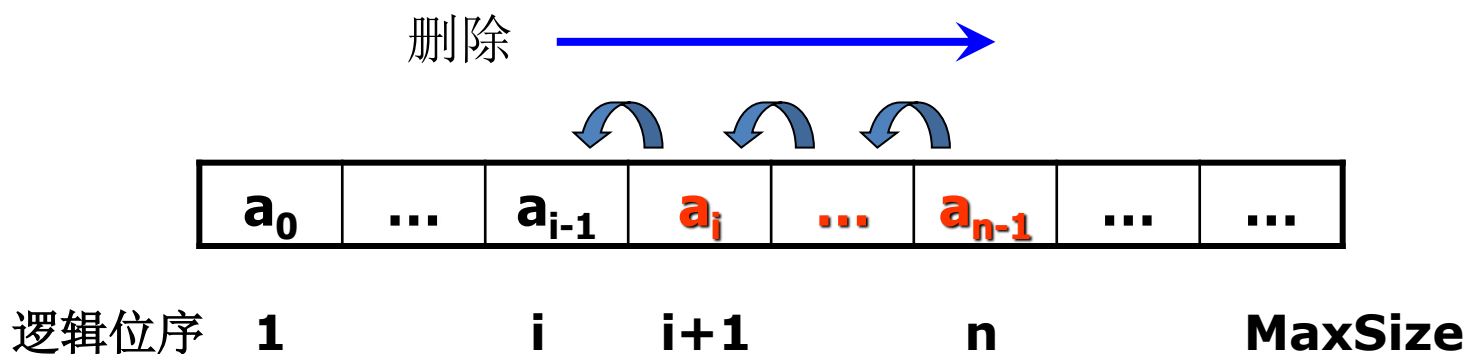
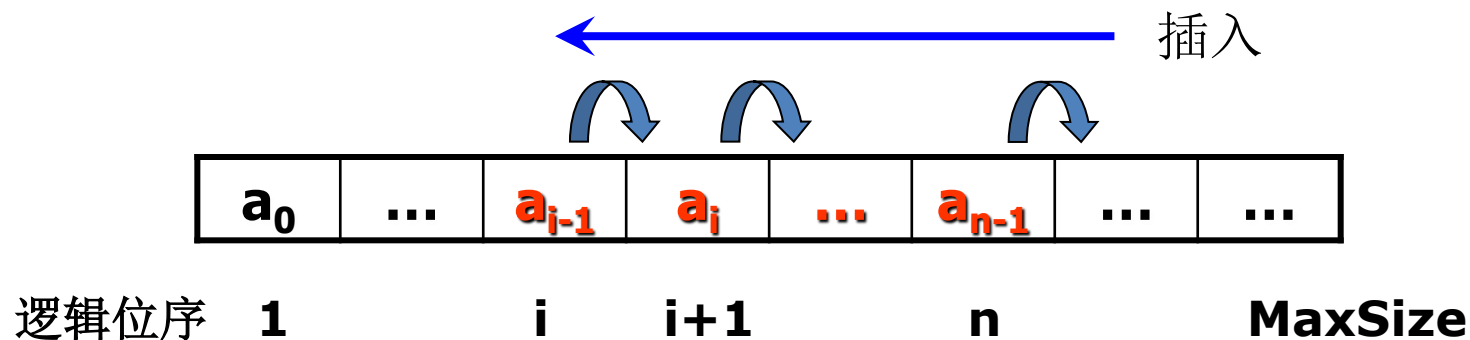
2.2 线性表的顺序表示和实现

```
#define MaxSize 100
typedef struct {
    ElemType data[MaxSize];
    int length;
} SqList;
void MergeList_Sq(SqList La, SqList Lb, SqList &Lc) {
    int i=0, j=0, k=0;
    InitList (Lc);
    while (i<=La.length-1 && j<=Lb.length-1) {
        if (La.data[i]<=Lb.data[j]) Lc.data[k++]=La.data[i++];
        else Lc.data[k++]=Lb.data[j++]; }
    while (i<=La.length-1) Lc.data[k++]=La.data[i++];
    while (j<=Lb.length-1) Lc.data[k++]=Lb.data[j++];
    Lc.length = k;
}
```

第2章 线性表

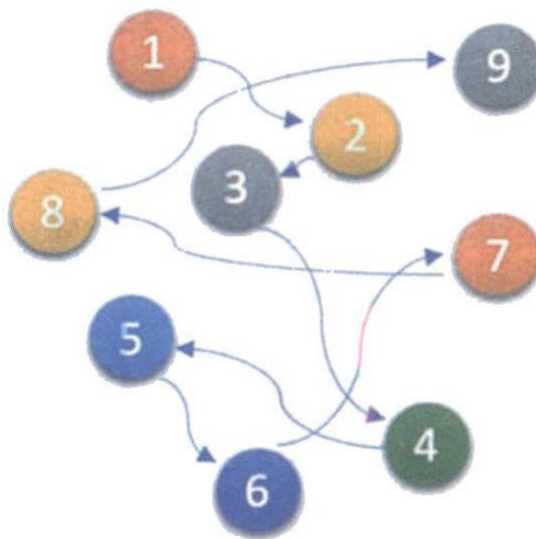
- 2.1 线性表的类型定义
- 2.2 线性表的顺序表示和实现
- 2.3 线性表的链式表示和实现
 - 2.3.1 线性链表
- 2.4 一元多项式的表示及相加

2.3.1 线性表的链式存储—链表



2.3.1 线性表的链式存储—链表

□ **特点：**在内存中用一组任意的存储单元来存储线性表的数据元素，用每个数据元素所带的指针来确定其后继元素的存储位置。这两部分信息组成数据元素的存储映像，称作**结点**。



2.3.1 线性表的链式存储—链表

在每个结点中除包含有数据域外,只设置一个指针域,用以指向其后继结点,这样构成的链接表称为线性单向链接表,简称**单链表**;

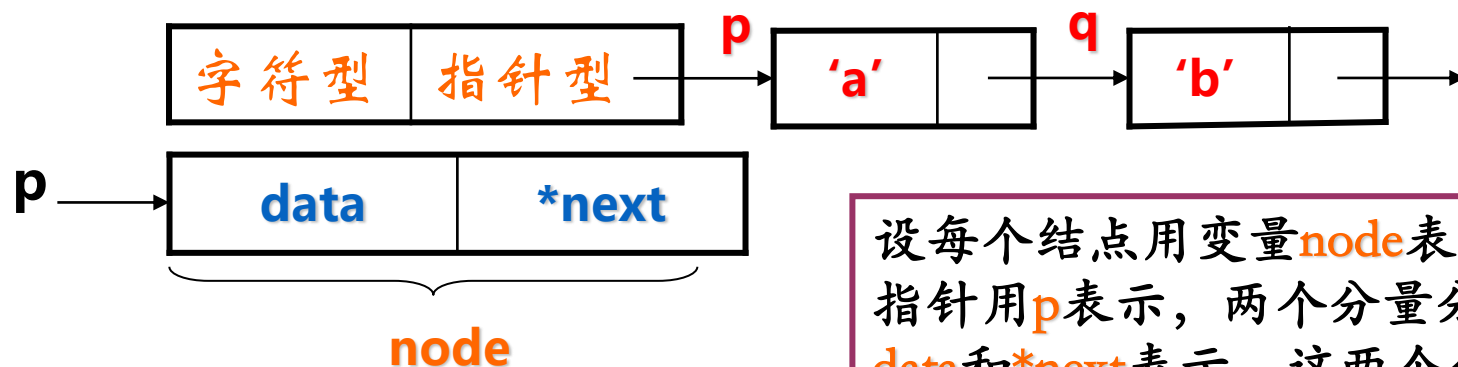


●**结点**：数据域 + 指针域（链域）

- 链式存储结构：n个结点链接成一个链表
- 线性链表（单链表）：链表的每个结点只包含一个指针域

2.3.1 线性表的链式存储—链表

以26个字母的链表为例，每个结点都有两个分量：

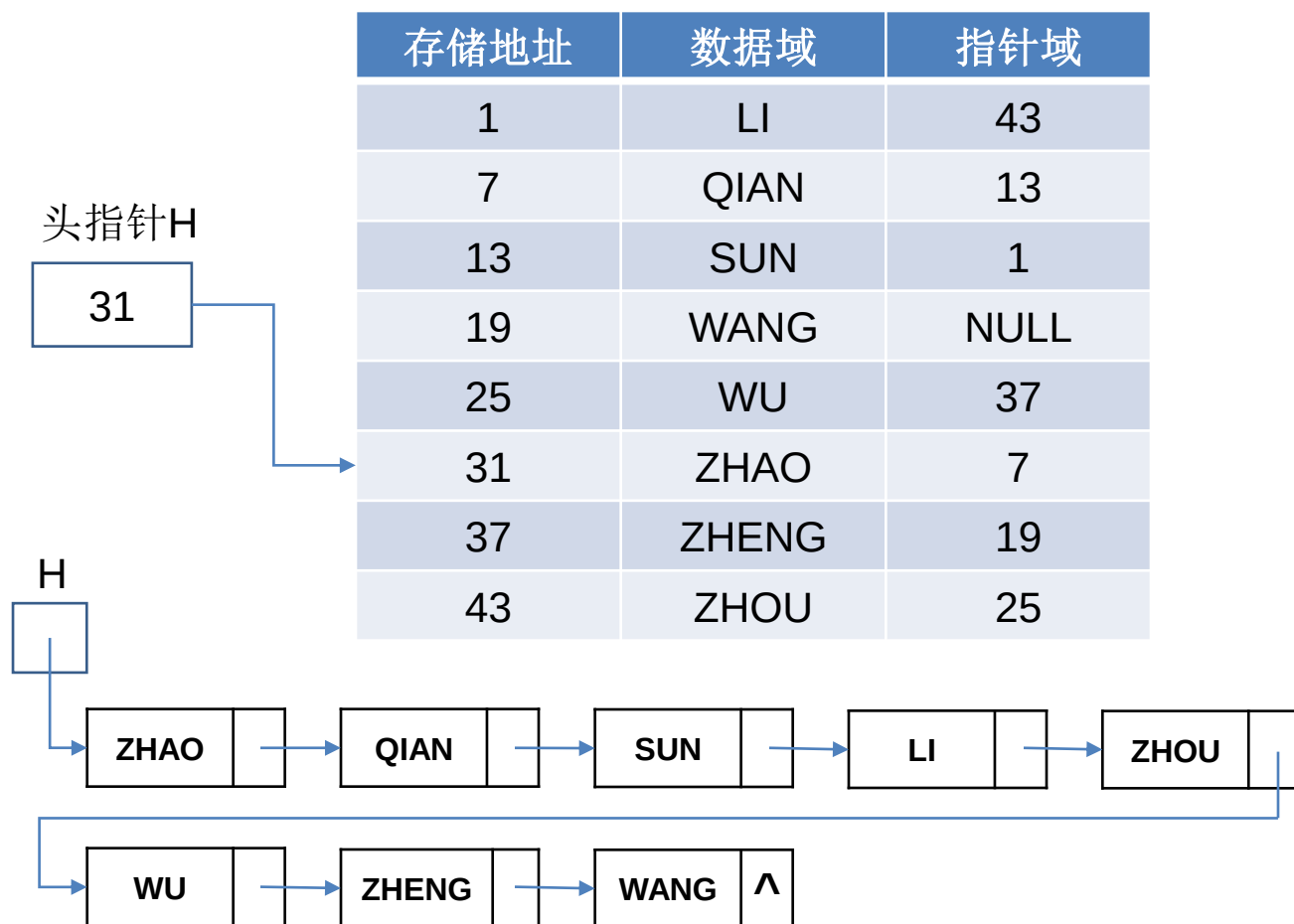


设每个结点用变量`node`表示，其指针用`p`表示，两个分量分别用`data`和`*next`表示，这两个分量如何赋值？

- 方式1： 直接表示为 `node.data = 'a'; node.next = q`
- 方式2： `p`指向结点首地址，然后 `p->data = 'a'; p->next = q;`
- 方式3： `p`指向结点首地址，然后 `(*p).data = 'a'; (*p).next = q`

2.3.1 线性表的链式存储—链表

例： (ZHAO, QIAN, SUN, LI, ZHOU, WU, ZHENG, WANG)



2.3.1 线性表的链式存储—链表

设 p 为指向链表的第 i 个元素的指针,则第 i 个元素的数据域写为 $p \rightarrow data$, 指针域写为 $p \rightarrow next$ 。



a_i 的值 a_{i+1} 的地址

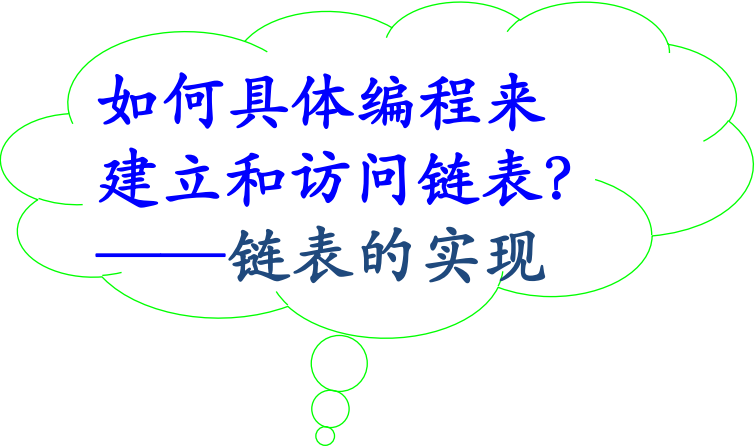
2.3.1 线性表的链式存储—链表

用结构类型和指针来表示顺序结构

```
typedef struct Lnode {  
    ElemType    data;    //数据域  
    struct Lnode *next;  //指针域  
}Lnode, *LinkList;      // *LinkList为Lnode类型的指针
```

Q1: 第一行的**Lnode** 与最后一行的**Lnode**是不是一回事?

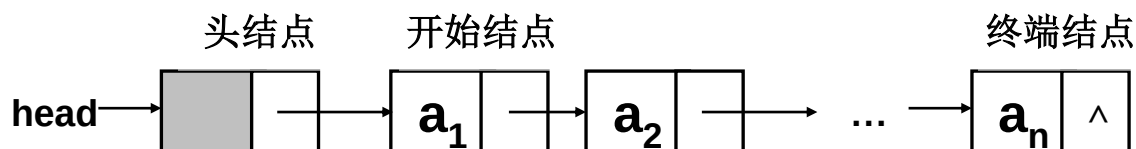
A1: 不是。前者**Lnode**是结构名，后者**Lnode**是对整个struct类型的一种“缩写”，是一种“**新定义名**”，它只是对现有类型名的补充，而不是取代。



如何具体编程来
建立和访问链表?
——链表的实现

2.3.1 线性表的链式存储—链表

在线性表的链式存储中,为了便于插入和删除算法的实现,每个链表带有一个头结点,并通过头结点的指针惟一标识该链表。



带头结点单链表示意图

- 说明：头结点的next域指向链表中的第一个数据元素结点。
- 对于头结点数据域的处理：
 - a.加特殊信息
 - b.置空
 - c.如数据域为整型，则在该处存放链表长度信息

2.3.1 线性表的链式存储—链表

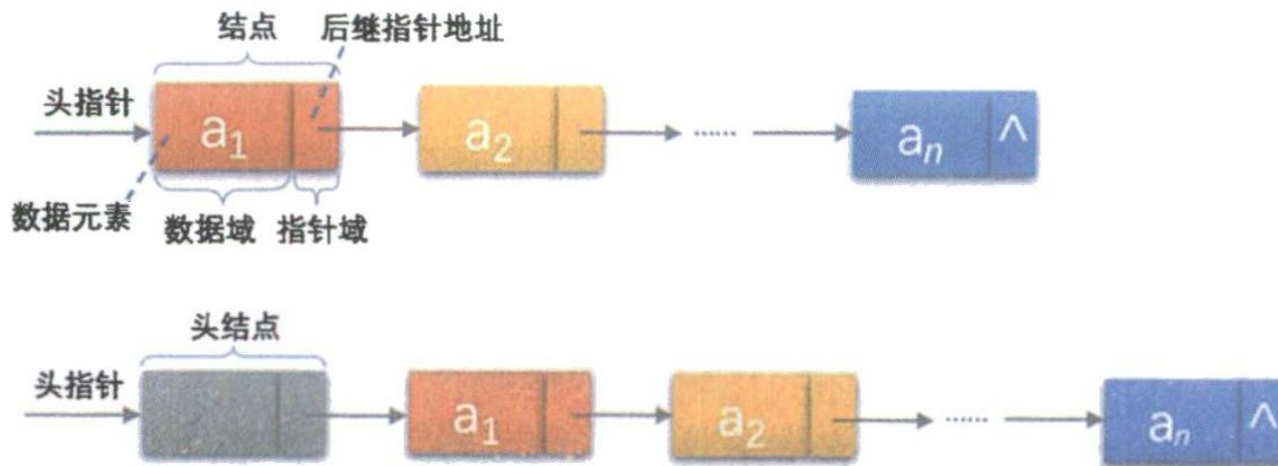
头指针

- 头指针是指链表指向第一个结点的指针，若链表有头结点，则是指向头结点的指针
- 头指针具有标志作用，所以常用头指针冠以链表的名字
- 无论链表是否为空，头指针均不为空。头指针是链表的必要元素

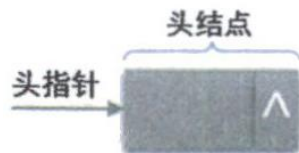
头结点

- 头结点是为了操作的统一和方便而设立的，放在第一元素的结点之前，其数据域一般无意义（也可存放链表的长度）
- 有了头结点，对在第一元素结点前插入结点和删除第一结点，其操作与其他结点的操作就统一了
- 头结点不一定是链表必需要素

2.3.1 线性表的链式存储—链表

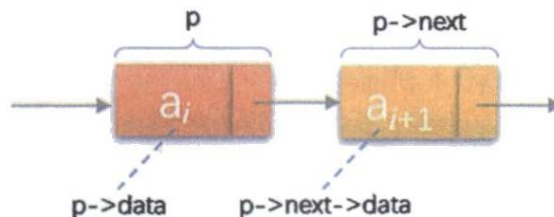


空链表:



L->next=NIL

后续节点访问:



无论链表是否为空，其头指针是指向头结点的非空指针，因此对空表与非空表的处理也就统一了，简化了链表操作的实现

2.3.1 线性表的链式存储—链表

基本运算

- 取元素 p29 算法2.8
- 插入元素 p30 算法2.9
- 删除元素 p30 算法2.10
- 建立链表 p30~p31 算法2.11
- 有序链表的合并 p31 算法2.12
- 查找（按值查找）
- 求长度
- 集合的并运算

2.3.1 线性表的链式存储—链表

(1)取元素

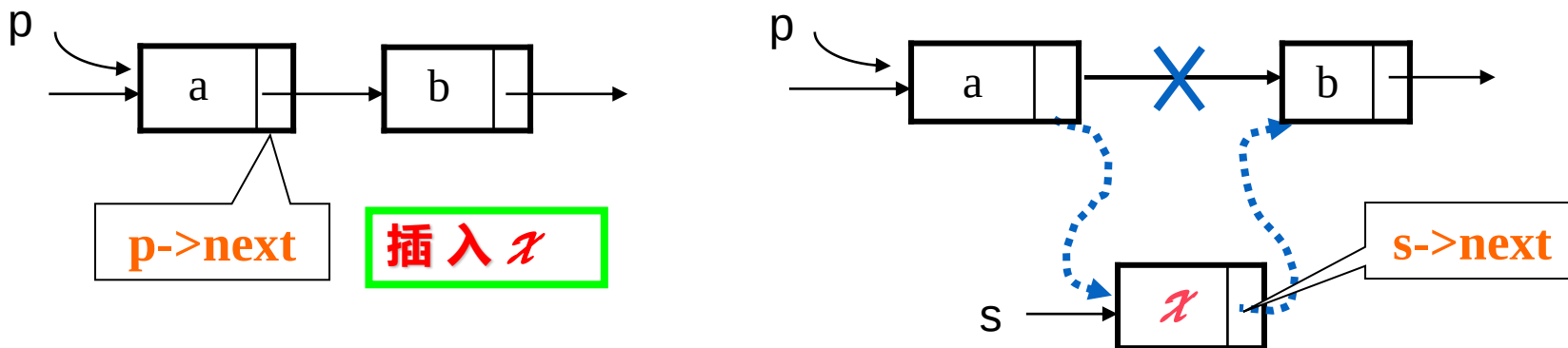
从链表的头指针出发，顺链域next逐个结点往下搜索，直至找到第i个结点为止 (j=i)

```
Status GetElem_L(LinkList L,int i,ElemType &e) {  
    LinkList p;  
    p=L->next; int j=1;  
    while (p && j<i) { p=p->next; ++j; }  
    if (!p || j>i) return ERROR;  
    e=p->data;  
    return OK;  
}
```

2.3.1 线性表的链式存储—链表

(2)插入元素

在链表中插入一个元素 x 的示意图如下：



链表插入的核心语句：

Step 1: $s \rightarrow next = p \rightarrow next;$

Step 2: $p \rightarrow next = s;$

x 结点的生成方式：

$s = (\text{node}^*) \text{malloc}(m);$

$s \rightarrow data = x;$

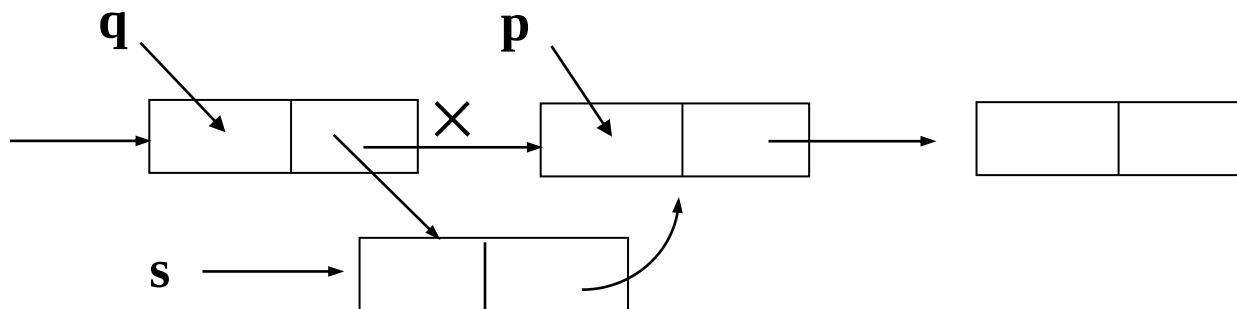
$s \rightarrow next = ?$

思考：Step1和Step2能互换么？

2.3.1 线性表的链式存储—链表

前插结点：设 p 指向链表中某结点， s 指向待插入的值为 x 的新结点，将 s 插入到 p 的前面。

思路：首先要找到 p 的前驱 q ，然后再完成在 q 之后插入 s ，设单链表头指针为 L



$q = L;$

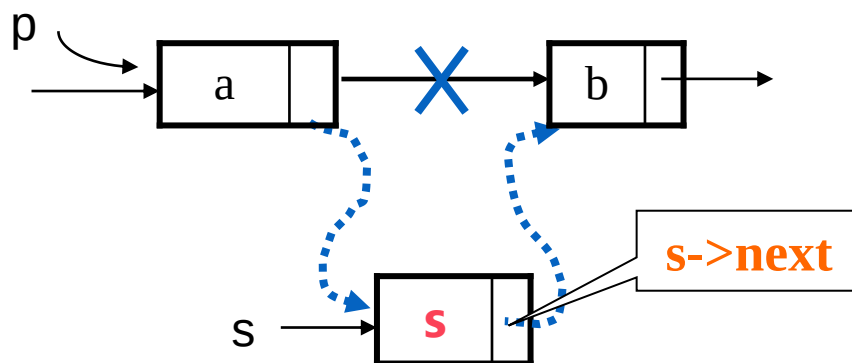
$\text{while } (q \rightarrow \text{next} \neq p) \quad q = q \rightarrow \text{next};$ /*找 p 的直接前驱*/

$s \rightarrow \text{next} = q \rightarrow \text{next};$

$q \rightarrow \text{next} = s;$

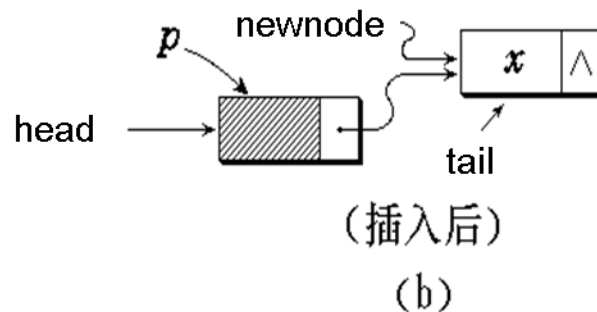
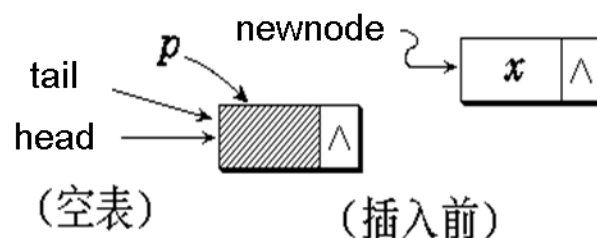
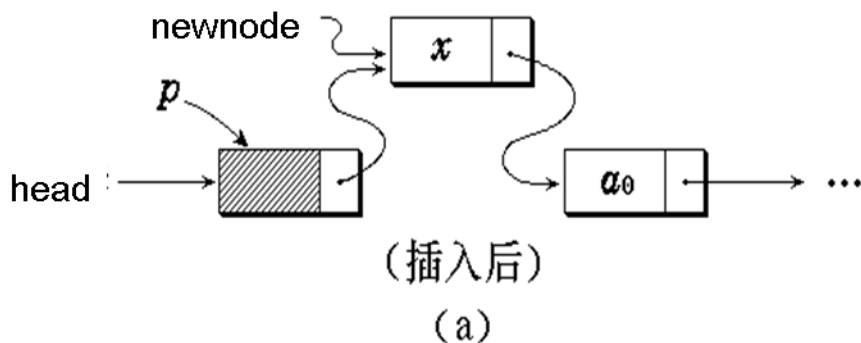
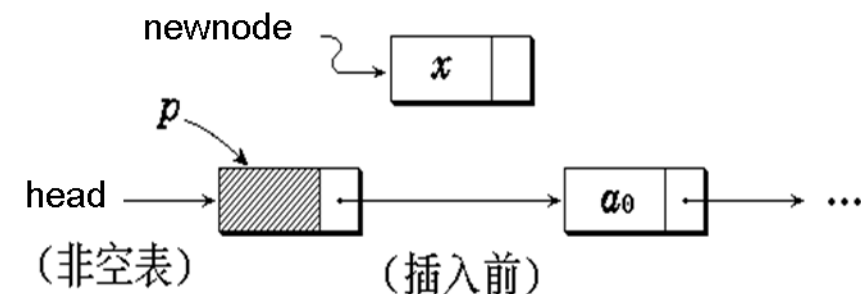
2.3.1 线性表的链式存储—链表

```
Status ListInsert_L(LinkList &L, int i, ElemType e) {  
    LinkList p, s;  
    p=L;    int j=0;  
    while (p && j<i-1) { p=p->next; ++j;}  
    if (!p || j> i-1) return ERROR;  
    s = (LinkList) malloc( sizeof (LNode));  
    s->data = e;  
    s->next = p->next;  
    p->next = s;  
    return OK;  
}
```



2.3.1 线性表的链式存储—链表

在带头结点的单链表第一个结点前插入新结点

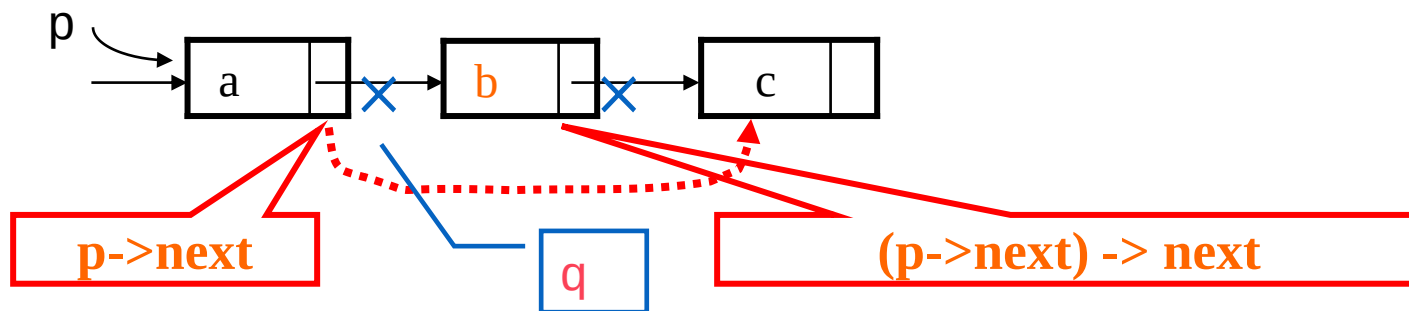


```
newnode→next = p→next;  
if ( p→next ==NULL ) tail = newnode;  
p→next = newnode;
```

2.3.1 线性表的链式存储—链表

(3)删除元素

设 p 指向单链表中某结点，删除 $*p$ 。操作示意图如图所示。要实现对结点 $*p$ 的删除，首先要找到 $*p$ 的前驱结点 $*q$ ，然后完成指针的操作即可。



删除动作的核心语句（要借助辅助指针变量 q ）：

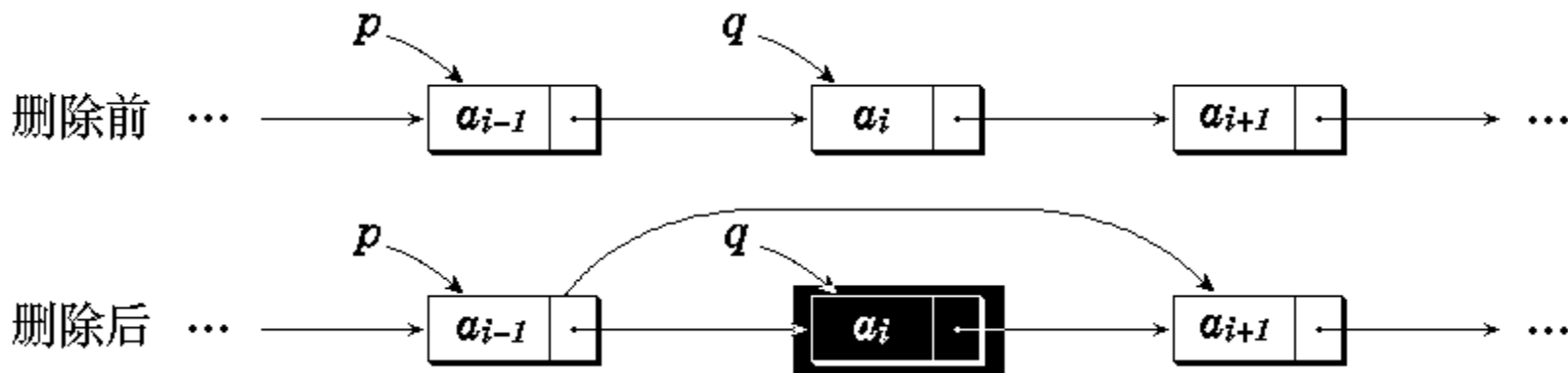
```
q = p->next;      //首先保存b的指针，靠它才能找到c；  
p->next=q->next;  //将a、c两结点相连，淘汰b结点；  
free(q);          //彻底释放b结点空间
```

思考：省略`free(q)`语句行不行？

2.3.1 线性表的链式存储—链表

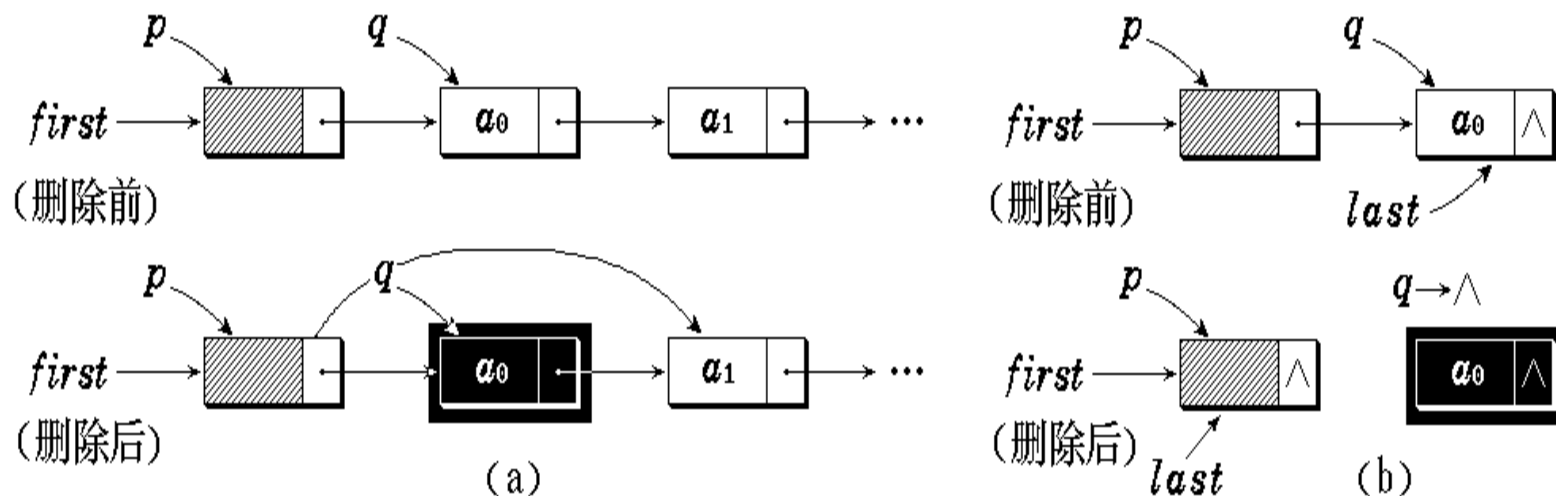
(3)删除元素

```
Status ListDelete_L(LinkList &L, int i, ElemType &e) {  
    LinkList p, q;  
    p=L;    int j=0;  
    while ( p->next && j<i-1 ) { p=p->next; ++j;}  
    if (!(p->next) || j> i-1) return ERROR;  
    q=p->next; p->next = q->next;  
    e=q->data; free(q);  
    return OK;  
}
```



2.3.1 线性表的链式存储—链表

从带表头结点的单链表中删除第一个结点



```
q = p→next;  
p→next = q→next;  
delete q;  
if ( p→next == NULL ) last = p;
```

2.3.1 线性表的链式存储—链表

插入和删除

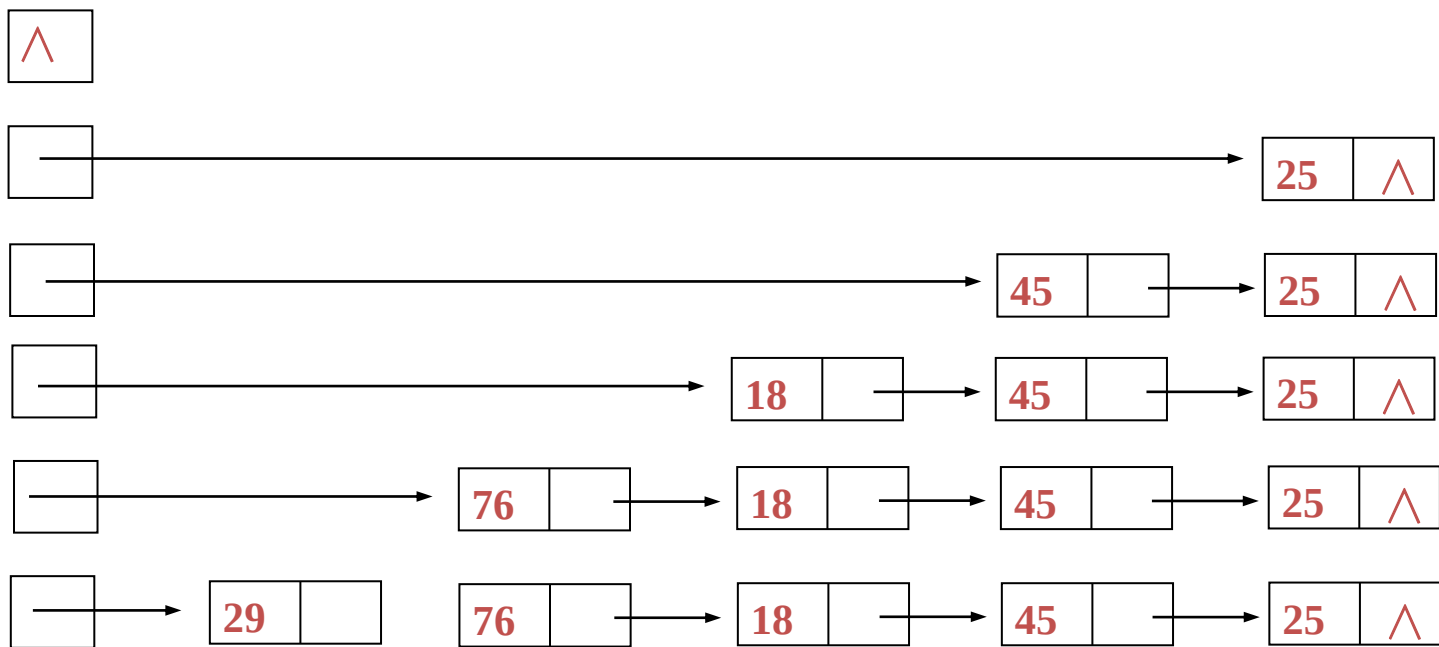
因线性链表不需要移动元素，只要修改指针，一般情况下时间复杂度为 $O(1)$ 。

如果要在单链表中进行前插或删除操作，因为要从头查找前驱结点，所耗时间复杂度将是 $O(n)$ 。

2.3.1 线性表的链式存储—链表

(4)建立单链表

□ 头插法建表：读取字符数组a中的字符,生成新结点,将读取的数据放到新结点的数据域中,然后将新结点插入到当前链表的表头上,直到结束为止:



线性表：（25,45,18,76,29）之链表的建立过程

2.3.1 线性表的链式存储—链表

(4)建立单链表

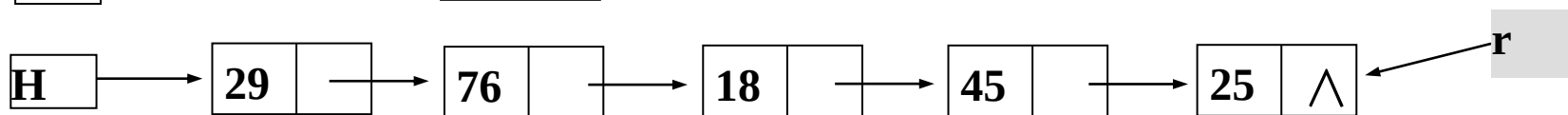
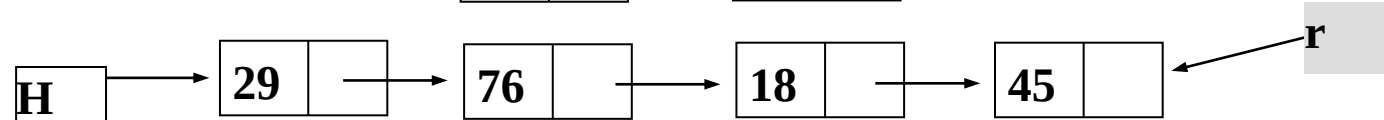
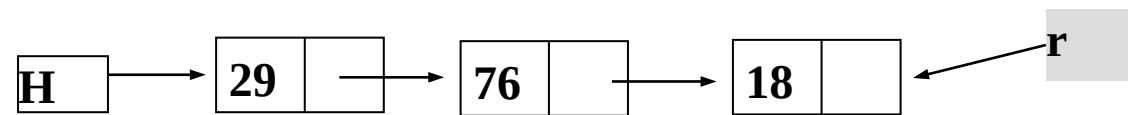
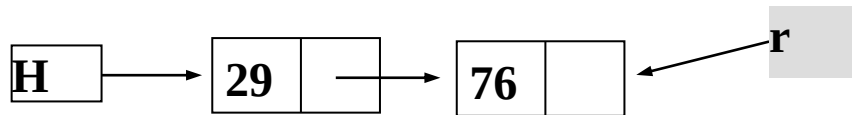
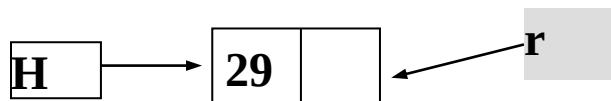
```
void CreateListL(LinkList &L,ElemType a[],int n) {  
    LNode *s;int i;  
    L=(LNode *)malloc(sizeof(LNode)); /*创建头结点*/  
    L->next=NULL;  
    for (i=0;i<n;i++){  
        s=(LNode *)malloc(sizeof(LNode));  
        /*创建新结点*/  
        s->data=a[i]; s->next=L->next;  
        /*将*s插入在原开始结点之前,头结点之后*/  
        L->next=s;  
    }  
}
```

2.3.1 线性表的链式存储—链表

□ 尾插法建表

将新结点插到当前链表的表尾上,为此必须增加一个尾指针 r ,使其始终指向当前链表的尾结点:

$H=NULL$ $r=NULL$ /*初始状态*/



线性表: (25,45,18,76,29) 之链表的建立过程

2.3.1 线性表的链式存储—链表

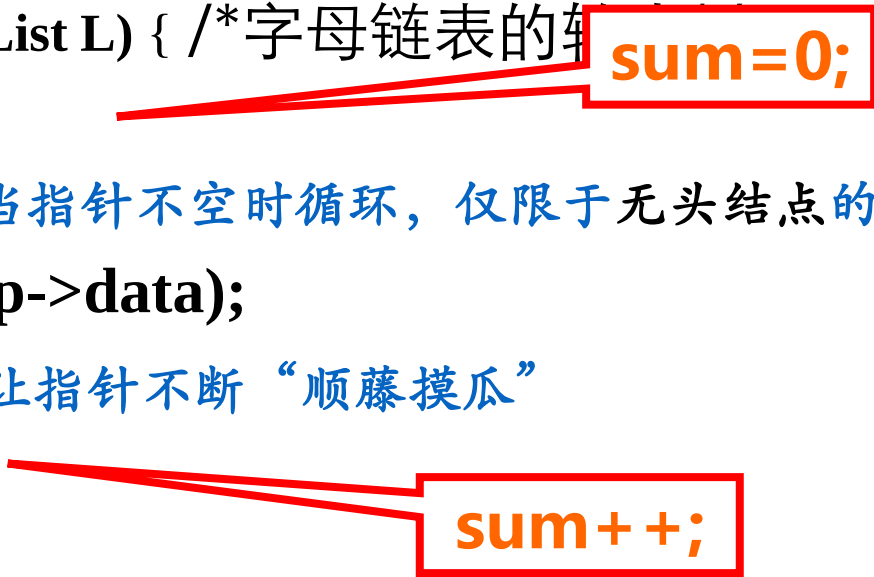
算法思路：初始状态，头指针 $H=NULL$ ，尾指针 $r=NULL$ ；按线性表中元素的顺序依次读入数据元素，不是结束标志时，申请结点，将新结点插入到 r 所指结点的后面，然后 r 指向新结点。

```
void CreateListR(LinkList &L,ElemType a[],int n){
    LNode *s,*r;int i;
    L=(LNode *)malloc(sizeof(LNode)); /*创建头结点*/
    r=L; /*r始终指向终端结点,开始时指向头结点*/
    for (i=0;i<n;i++){
        s=(LNode *)malloc(sizeof(LNode)); /*创建新结点*/
        s->data=a[i];r->next=s; /*将*s插入*r之后*/
        r=s;
    }
    r->next=NULL; /*终端结点next域置为NULL*/
}
```

2.3.1 线性表的链式存储—链表

(5)显示单链表

```
void DispList(LinkList L) { /*字母链表的输出*/
    p = L;
    while (p) { //当指针不空时循环，仅限于无头结点的情况
        printf("%c",p->data);
        p=p->next; //让指针不断“顺藤摸瓜”
    }
}
```



sum=0;

sum++;

讨论：要统计链表中数据元素的个数，该如何改写？

2.3.1 线性表的链式存储—链表

(6)单链表的修改(或读取)

思路：要修改第*i*个数据元素，必须从头指针起一直找到该结点的指针*p*，然后才能执行 `p->data=new_value` 。

修改第*i*个数据元素的操作函数可写为：

```
Status GetElem_L(LinkList L, int i, ElemType &e) {  
    p=L->next; j=1;    //带头结点的链表  
    while(p&& j<i) {p=p->next; ++j;}  
    if( !p ) return ERROR;  
    p->data =e; //若是读取则为： e=p->data;  
    return OK;  
} // GetElem_L
```

缺点：想寻找单链表中第*i*个元素，只能从头指针开始逐一查询（顺藤摸瓜），无法随机存取。

2.3.1 线性表的链式存储—链表

(7)初始化带头结点的线性表

该运算建立一个空的单链表，即创建一个头结点。

```
void InitList(LinkList &L)
{
    L=(LNode *)malloc(sizeof(LNode)); /*创建头结点*/
    L->next=NULL;
}
```

2.3.1 线性表的链式存储—链表

(8)销毁带头结点的线性表

释放单链表L占用的内存空间。即逐一释放全部结点的空间。

```
void DestroyList(LinkList &L) {  
    LNode *p=L,*q=p->next;  
    while (q!=NULL)  
    {    free(p);  
        p=q;q=p->next;  
    }  
    free(p);  
}
```

2.3.1 线性表的链式存储—链表

(9)判断带头结点的线性表是否为空表

```
int ListEmpty(LinkList L) {  
    return(L->next==NULL);  
}
```

2.3.1 线性表的链式存储—链表

(10)求带头结点的线性表长度

```
int ListLength(LinkList L) {  
    LNode *p=L;int i=0;  
    while (p->next!=NULL) {  
        i++;  
        p=p->next;  
    }  
    return(i);  
}
```

2.3.1 线性表的链式存储—链表

(11)查找带头结点线性表中的元素

```
int LocateElem(LinkList L,ElemType e) {  
    LNode *p=L->next;int n=1;  
    while (p!=NULL && p->data!=e) {  
        p=p->next; n++;  
    }  
    if (p==NULL) return(0);  
    else return(n);  
}
```

例

合并有序链表

算法要求：

已知：线性表 A 和 B，分别由单链表 L_a 和 L_b 存储，其中数据元素按值非递减有序排列（即已经有序）；

要求：将 A 和 B 归并为一个新的线性表 C，C 的数据元素仍按值非递减排列。设线性表 C 由单链表 L_c 存储。

假设：A = (3, 5, 8, 11)，B = (2, 6, 8, 9, 11)

预测：合并后的 C = (2, 3, 5, 6, 8, 8, 9, 11, 11)

例

算法主要包括搜索、比较、插入三个操作：

搜索：需要设立三个指针来指向La、Lb和Lc链表；

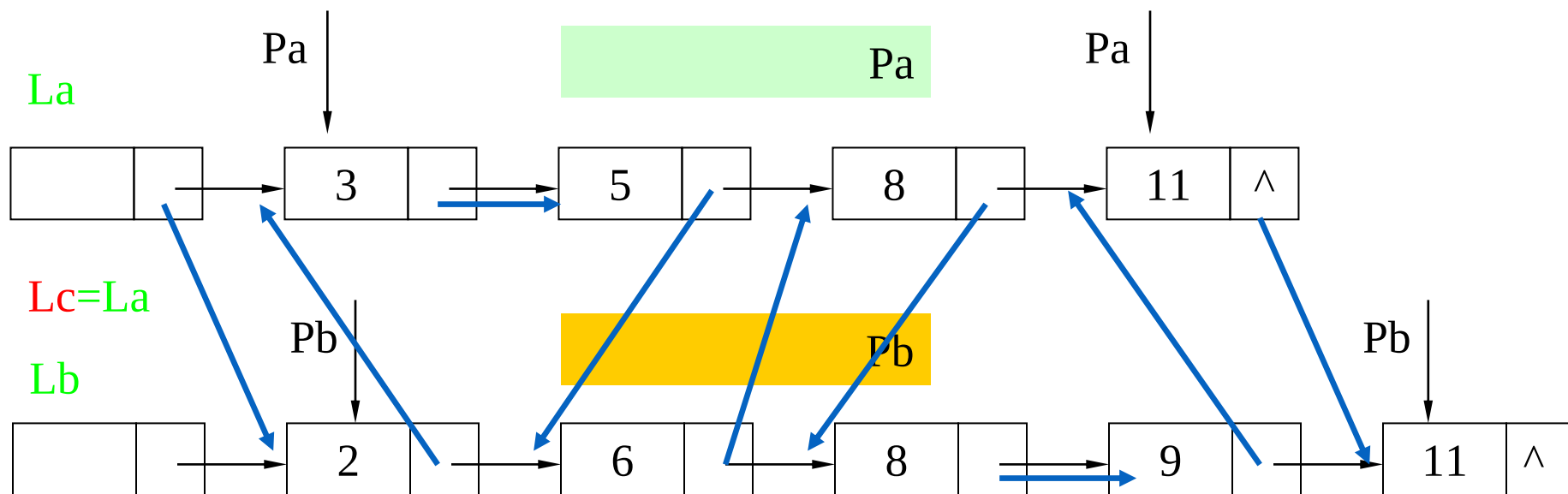
比较：比较La和Lb表中结点数据的大小；

插入：将La和Lb表中数据较小的结点插入新链表Lc。

请注意链表的特点，仅改变指针便可实现数据的移动，即
“数据不动，指针动”

例

P_a 、 P_b 用于搜索 L_a 和 L_b ， P_c 指向新链表当前结点。



例

```
void MergeList_L(LinkList &La, LinkList &Lb, LinkList &Lc) {
    LinkList pa, pb, pc;
    pa = La->next; pb = Lb->next;
    Lc = pc = La;
    while (pa && pb) {
        if (pa->data <= pb->data) {
            pc->next = pa; pc = pa; pa = pa->next;
        }
        else {pc->next = pb; pc = pb; pb = pb->next;}
    }
    pc->next = pa ? pa : pb;
    free(Lb);
}
```

不用Lc，直接把La表插到Lb表中；或者把Lb表插到La表中，怎么修改？

例

合并 $A=A \cup B$

```
void UnionList_L(LinkList &La, LinkList Lb) {
    LinkList p, q, first;  int x;
    first = La->next;  p=Lb->next;
    while (p) {
        x=p->data;  p=p->next;  q=first;
        while (q && q->data !=x)  q=q->next;
        if (!q) {
            q=(LinkList)malloc(sizeof(LNode));
            q->data = x;  q->next = La->next;
            La->next = q;
        }
    }
}
```

例

**例 设 $C=\{a_1, b_1, a_2, b_2, \dots, a_n, b_n\}$ 为一线性表,采用带头结点的hc单链表存放,编写一个算法,将其拆分为两个线性表,使得:
 $A=\{a_1, a_2, \dots, a_n\}, B=\{b_1, b_2, \dots, b_n\}$**

```
typedef struct LNode {  
    ElemType    data;    //数据域  
    struct Lnode *next;  //指针域  
}LNode, *LinkList;
```

例

```
void fun(LinkList hc, LinkList &ha, LinkList &hb) {
    LNode *p=hc->next,*ra,*rb;
    ha=hc;          /*ha的头结点利用hc的头结点*/
    ra=ha;          /*ra始终指向ha的末尾结点*/
    hb=(LNode *)malloc(sizeof(LNode)); /*创建hb头结点*/
    rb=hb;          /*rb始终指向hb的末尾结点*/
    while (p!=NULL) {
        ra->next=p;ra=p; /*将*p链到ha单链表末尾*/
        p=p->next;
        if (p!=NULL)
        {   rb->next=p;
            rb=p;   /*将*p链到hb单链表末尾*/
            p=p->next;
        }
    }
    ra->next=rb->next=NULL; /*两个尾结点的next域置空*/
}
```

例

下述哪一条是顺序存储结构的优点?()

-  ☒ A 存储密度大
- ☐ B 插入运算方便
- ☐ C 删除运算方便
- ☐ D 可方便地用于各种逻辑结构的存储表示

例

链表不具有的特点是()

- ☐ A 插入、删除不需要移动元素
- ☒ B 可随机访问任一元素
- ☐ C 不必事先估计存储空间
- ☐ D 所需空间与线性长度成正比

例

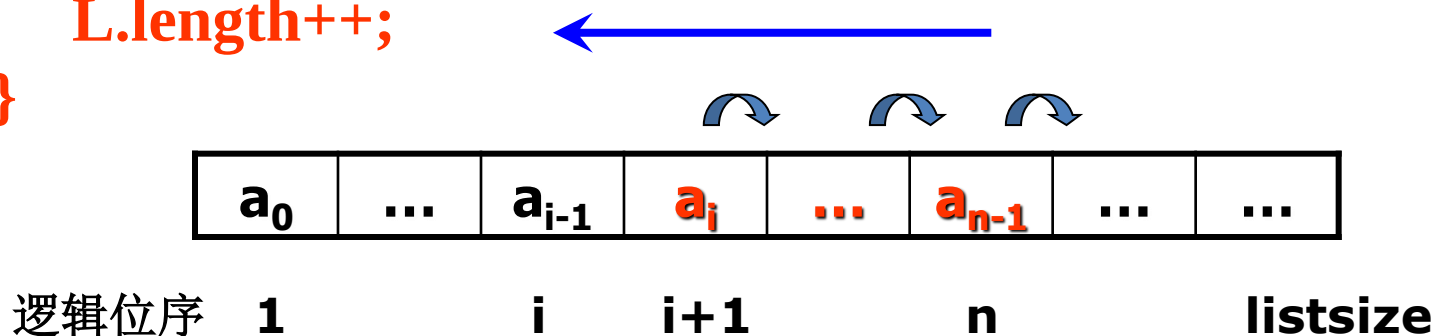
顺序表应用举例

例 设计一个算法,将 x 插入到一个有序(从小到大排序)的线性表(顺序存储结构即顺序表)的适当位置上,并保持线性表的有序性。

解:先通过比较在顺序表 L 中找到存放 x 的位置 i ,然后将 x 插入到 $L.elem[i]$ 中,最后将顺序表的长度增1。

例

```
void Insert(SqList &L, ElemType x) {  
    int i=0, j;  
    if (L.length >= L.listsize) return OVERFLOW;  
    while (i<L.length && L.elem[i]<x)      } 查找插入位置  
        i++;  
    for (j=L.length-1; j>=i; j--)          } 元素后移一个位置  
        L.elem[j+1]=L.elem[j];  
    L.elem[i]=x;  
    L.length++;  
}
```



例

例 已知长度为 n 的线性表 A 采用顺序存储结构,编写一个时间复杂度为 $O(n)$ 、空间复杂度为 $O(1)$ 的算法,该算法删除线性表中所有值为 $item$ 的数据元素。

解:用 k 记录顺序表 A 中等于 $item$ 的元素个数,边扫描 A 边统计 k ,并将不为 $item$ 的元素前移 k 个位置,最后修改 A 的长度。对应的算法如下:

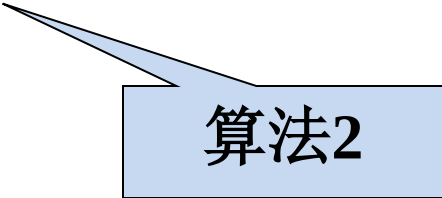
例

```
void delnode1(SqList &A,ElemType item)
{   int k=0,i; /*k记录值不等于item的元素个数*/
    for (i=0;i<A.length;i++)
        if (A.elem[i]!=item)
        {   A.elem[k]=A.data[i];
            k++; /*不等于item的元素增1*/
        }
    A.length=k; /*顺序表A的长度等于k*/
}
```

算法1：类似于
建顺序表

例

```
void delnode2(SqList &A,ElemType item)
{   int k=0,i=0; /*k记录值等于item的元素个数*/
    while (i<A.length)
    {   if (A.elem[i]==item)  k++;
        else A.elem[i-k]=A.elem[i]; /*当前元素前移k个位置*/
        i++;
    }
    A.length=A.length-k; /*顺序表A的长度递减*/
}
```



算法2

例

上述算法中只有一个while循环,时间复杂度为 $O(n)$ 。算法中只用了 i, k 两个临时变量,空间复杂度为 $O(1)$ 。

例

例 将顺序表 $(a_1, a_2, \dots, a_n)$ 重新排列为以 a_1 为界的两部分： a_1 前面的值均比 a_1 小， a_1 后面的值都比 a_1 大(这里假设数据元素的类型具有可比性，不妨设为整型)，操作前后如图所示。这一操作称为划分。 a_1 也称为基准。

25	15
30	10
20	20
60	25
10	30
35	60
15	35
.	.
.	.
.	.

例

基本思路：

从第二个元素开始到最后一个元素，逐一向后扫描：（1）当前数据元素 a_i 比 a_1 大时，表明它已经在 a_1 的后面，不必改变它与 a_1 之间的位置，继续比较下一个。

（2）当前结点若比 a_1 小，说明它应该在 a_1 的前面，此时将它上面的元素都依次向下移动一个位置，然后将它置入最上方。

例

```
void part(SeqList *L) {
    int i,j;
    datatype x,y;
    x=L->data[0];  /* 将基准置入 x 中*/
    for(i=1;i<=L->last;i++)
        if(L->data[i]<x) {
            y = L->data[i];
            for(j=i-1;j>=0;j--) /*移动*/
                L->data[j+1]=L->data[j];
            L->data[0]=y;
        }
}
```

正在答疑
